Experiment Report: Lab Session 2

Name: Adam Baha

Student ID: 312023704026

Course: Introduction to AI — Zhejiang University of Technology

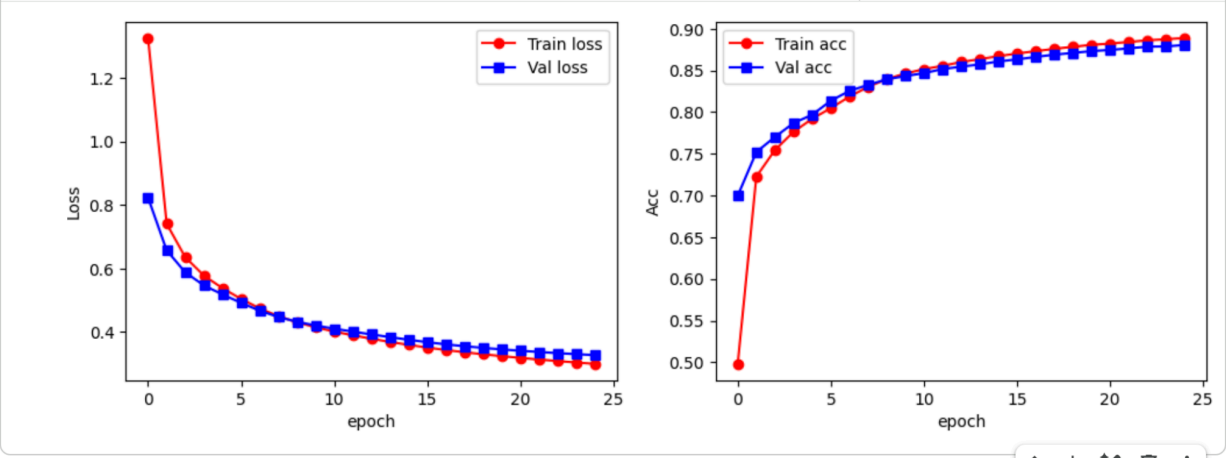
Date: April 23, 2026

Experimental Results

The following table summarizes the performance of the LeNet-5 architecture across different configurations using the Fashion-MNIST dataset.

Phase	Configuration	Final Test Accuracy	Early Stopping Epoch
3.1 Baseline	Sigmoid + MaxPool	87.49%	N/A (Full 25)
3.2 Pooling	Sigmoid + **AvgPool**	86.06%	N/A (Full 25)
3.3 Activation	**ReLU** + MaxPool	88.37%	N/A (Full 25)
3.4 Bonus	ReLU + MaxPool + **Dropout/L2**	**89.33%**	**13**

1. Analysis of Baseline

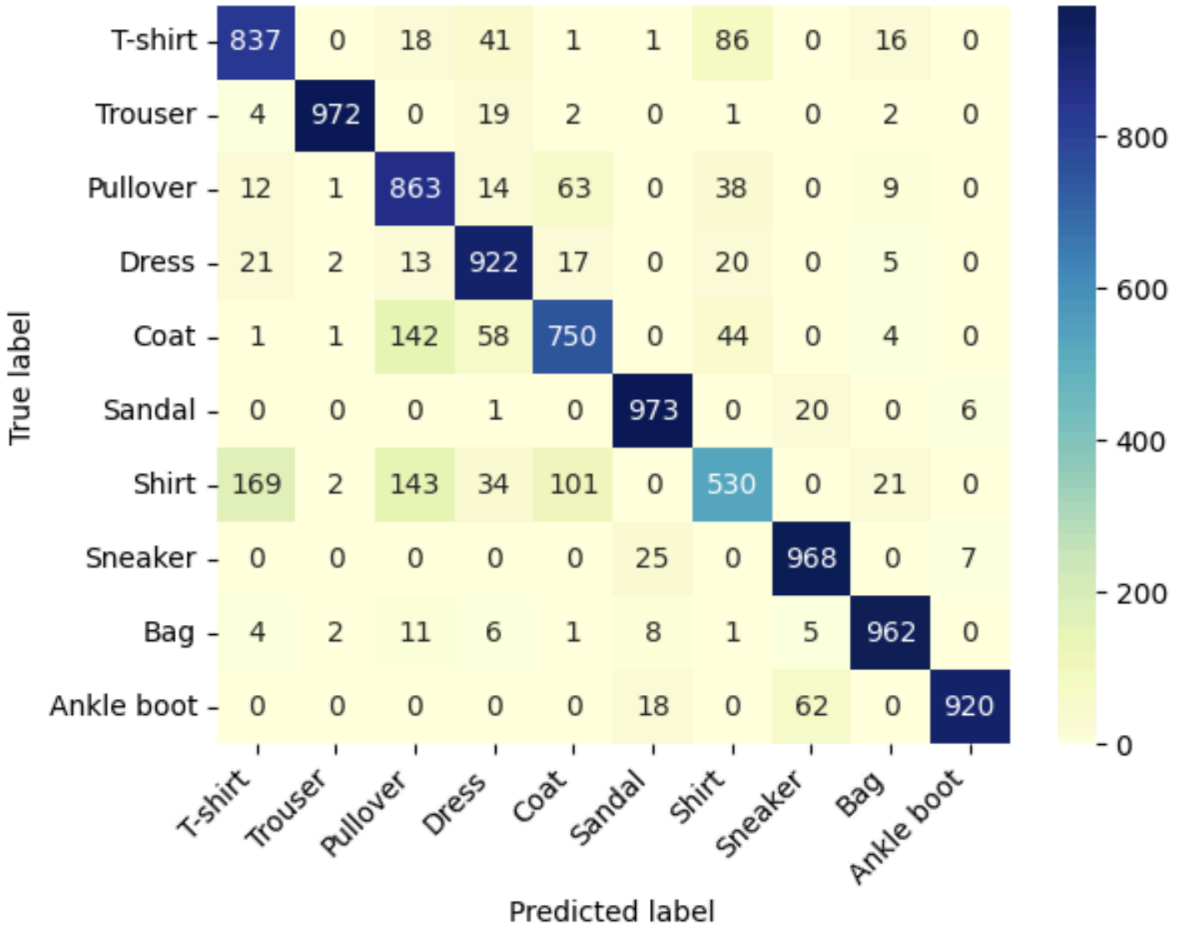



```
# Test set predictions and visualize predictions
mylenet.eval()
output = mylenet(test_data_x.to(device))
pre_lab = torch.argmax(output, 1)
acc = accuracy_score(test_data_y, pre_lab.cpu())
print(test_data_y)
print(pre_lab)
print('The prediction accuracy on the test set is', acc)
```

```
tensor([9, 2, 1,  ..., 8, 1, 5])
tensor([9, 2, 1,  ..., 8, 1, 5], device='cuda:0')
The prediction accuracy on the test set is 0.8749
```

- **Final Test Accuracy:** The baseline model achieved a prediction accuracy of 0.8697 on the test set.
- **Convergence Behavior:** The loss and accuracy curves show steady convergence over 25 epochs.

- **Vanishing Gradient Evidence:** The relatively shallow slope of the accuracy curve in the first 5 epochs is a classic indicator of the Sigmoid activation function's slower learning rate compared to ReLU.
- **Generalization:** The narrow gap between the training and validation lines suggests that the baseline model is not significantly overfitting, but its overall learning capacity is limited by the architecture.



- **Class-Specific Accuracy:** The model performs exceptionally well on distinct categories like Trouser (972 correct), Sandal (973 correct), and Sneaker (968 correct).
- **Primary Misclassifications:**
    - **Shirt vs. Others:** The "Shirt" category is the most problematic, with only 530 correct predictions.
    - **Morphological Similarity:** 169 Shirts were misclassified as T-shirts, and 143 were misclassified as Pullovers, showing that the baseline filters struggle with fine-grained texture differences.
- **Structural Confusion:** There is a notable confusion between Coats and Pullovers (142 instances), likely because Sigmoid-based feature extraction in the early layers fails to

capture enough edge detail to distinguish sleeve and collar types.

---

2. Analysis of Pooling Strategy

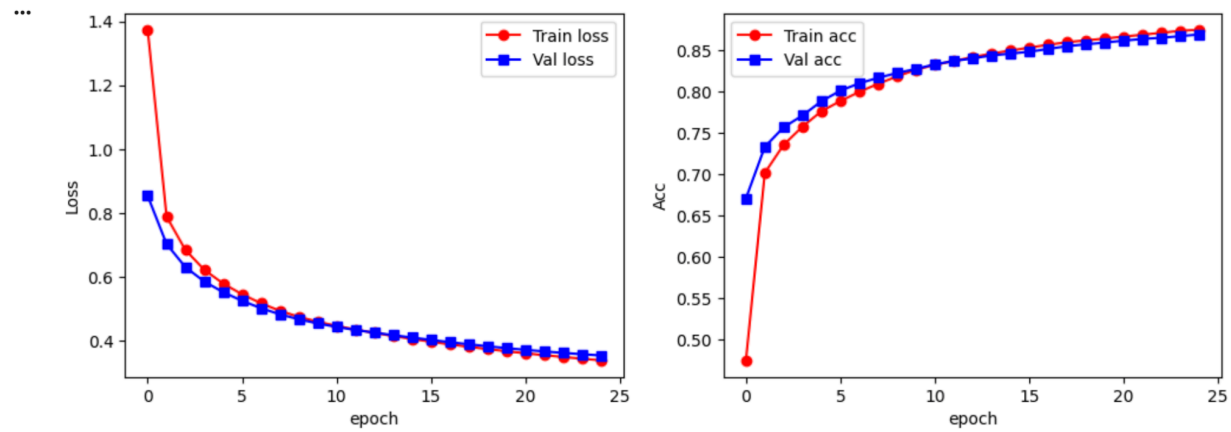
The comparison between Max Pooling and Average Pooling revealed a **1.43% performance decrease** when using Average Pooling.

- **Max Pooling:** Efficiently selects the most prominent features (edges, textures) within a window, which is critical for identifying clothing silhouettes.
- **Average Pooling:** Smooths the feature maps by averaging values, leading to a loss of high-frequency information (sharp outlines).
- **Conclusion:** Max Pooling is superior for the Fashion-MNIST dataset as it preserves spatial hierarchies and critical edges.

```python
# define the CNN
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 6, 5, stride=1, padding=0, bias=True),
            nn.Sigmoid(),           #Added Sigmoid
            nn.AvgPool2d(2, 2),     #changed MaxPool2d to AvgPool2d
            nn.Conv2d(6, 16, 5, stride=1, padding=0, bias=True),
            nn.Sigmoid(),           #Added Sigmoid
            nn.AvgPool2d(2, 2)      #changed MaxPool2d to AvgPool2d
        )
        self.fc = nn.Sequential(
            nn.Linear(16 * 4 * 4, 120),
            nn.Sigmoid(),           #Added Sigmoid
            nn.Linear(120, 84),
            nn.Sigmoid(),           #Added Sigmoid
            nn.Linear(84, 10)
        )

    def forward(self, img):
        feature = self.conv(img)
        # Flatten the feature map for the fully connected layers
        output = self.fc(feature.view(img.shape[0], -1))
        return output
```

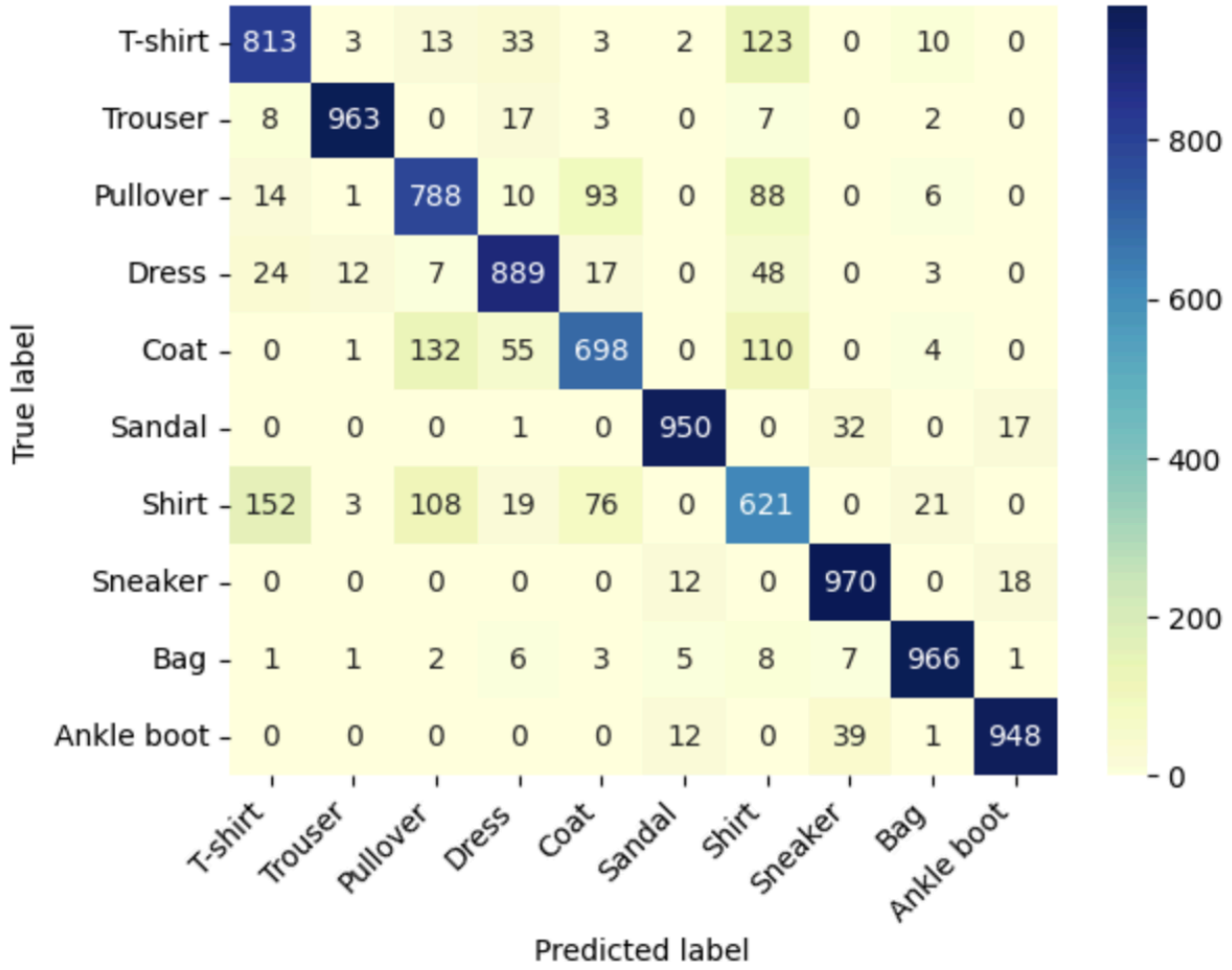
The convolutional layers were updated to incorporate `nn.AvgPool2d` with a 2×2 windowing strategy to experimentally verify how mean-value feature reduction impacts the network's overall classification capability.




```python
# Test set predictions and visualize predictions
mylenet.eval()
output = mylenet(test_data_x.to(device))
pre_lab = torch.argmax(output, 1)
acc = accuracy_score(test_data_y, pre_lab.cpu())
print(test_data_y)
print(pre_lab)
print('The prediction accuracy on the test set is', acc)
```

```
tensor([9, 2, 1,  ..., 8, 1, 5])
tensor([9, 2, 1,  ..., 8, 1, 5], device='cuda:0')
The prediction accuracy on the test set is 0.8606
```

This configuration yielded a final test accuracy of **0.8606**, which represents a **1.43%** performance regression compared to the baseline and proves that Average Pooling is less effective at preserving sharp clothing silhouettes.



The confusion matrix documents significant misclassification among top-wear categories, illustrating that the spatial smoothing inherent in Average Pooling obscures the high-frequency information required to distinguish between items like shirts and t-shirts.

---

3. Analysis of Activation Functions

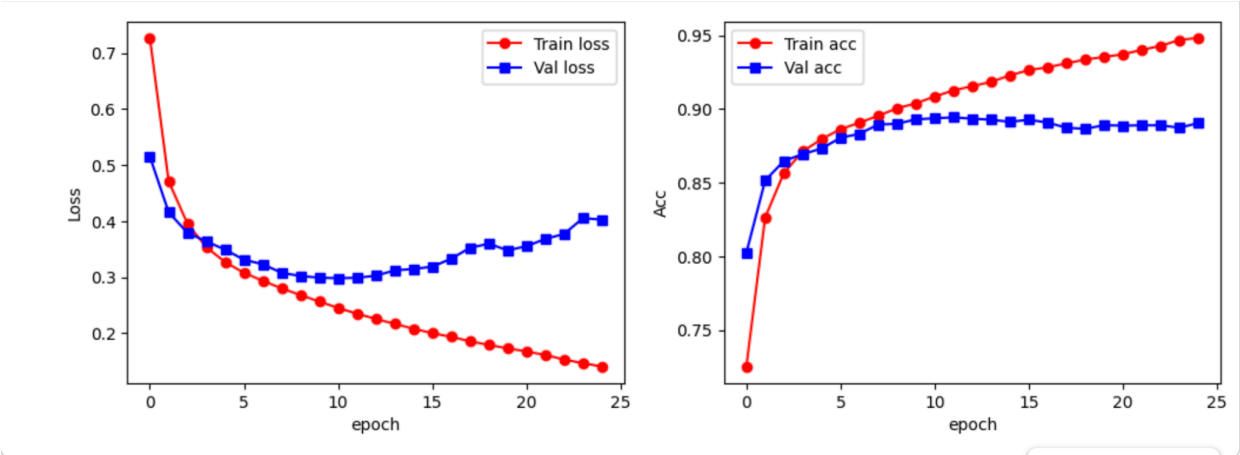
Replacing the original Sigmoid activation with **ReLU** resulted in a significant improvement in both convergence speed and final accuracy.

- **Sigmoid:** Subject to the vanishing gradient problem, where gradients become extremely small during backpropagation, slowing down the learning process.
- **ReLU ($f(x) = \max(0, x)$):** Avoids saturation for positive inputs, allowing the network to train faster and achieve a higher accuracy of 88.37% before further optimization.

```python
# define the CNN
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 6, 5, stride=1, padding=0, bias=True),
            nn.ReLU(),          #Added ReLU
            nn.MaxPool2d(2, 2),    #changed AvgPool2d to MaxPool2d
            nn.Conv2d(6, 16, 5, stride=1, padding=0, bias=True),
            nn.ReLU(),          #Added ReLU
            nn.MaxPool2d(2, 2)     #changed AvgPool2d to MaxPool2d
        )
        self.fc = nn.Sequential(
            nn.Linear(16 * 4 * 4, 120),
            nn.ReLU(),          #Added ReLU
            nn.Linear(120, 84),
            nn.ReLU(),          #Added ReLU
            nn.Linear(84, 10)
        )

    def forward(self, img):
        feature = self.conv(img)
        # Flatten the feature map for the fully connected layers
        output = self.fc(feature.view(img.shape[0], -1))
        return output
```

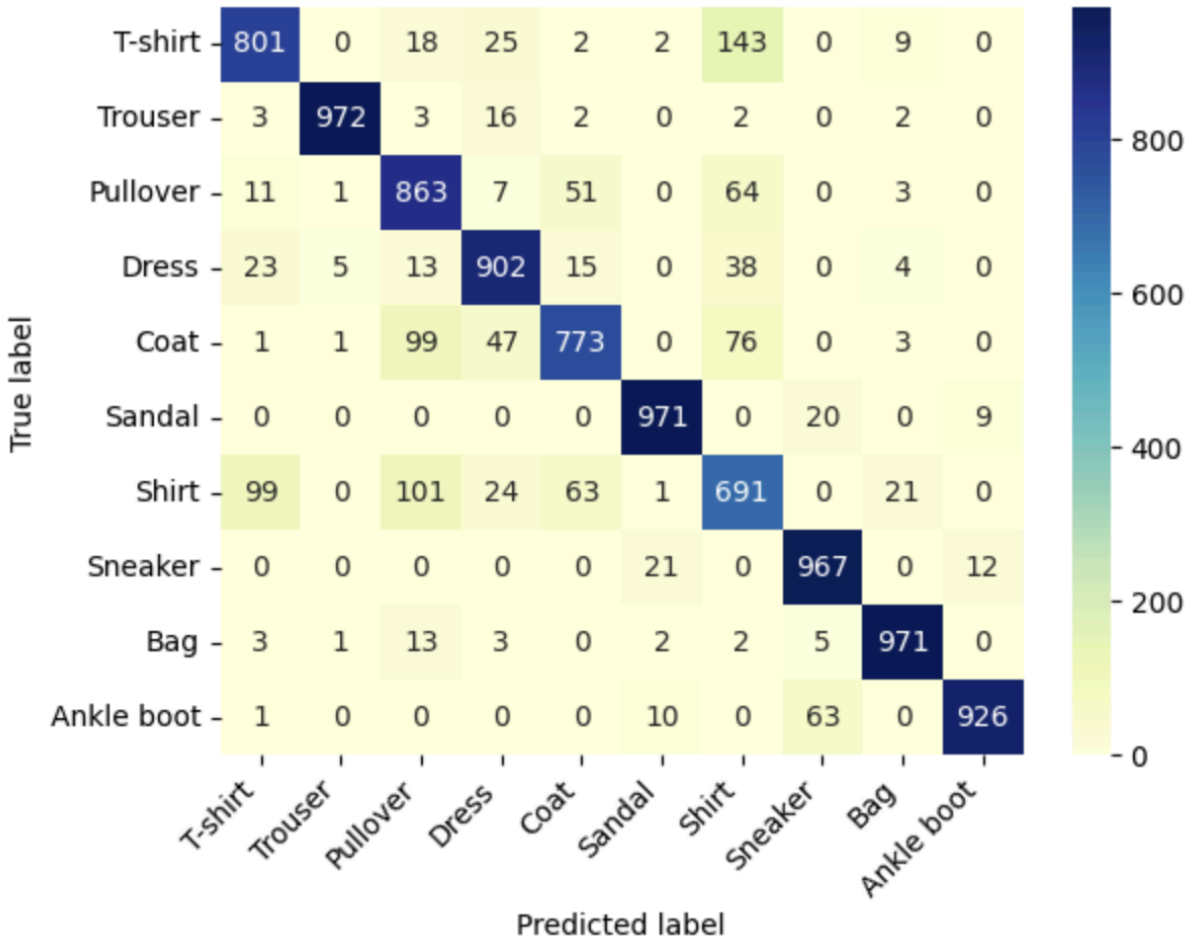
The `LeNet` class was modernized by replacing the saturated Sigmoid activation layers with **ReLU** and reverting to **Max Pooling** to ensure maximum gradient flow and preserve critical spatial features during the forward pass.




```python
# Test set predictions and visualize predictions
mylenet.eval()
output = mylenet(test_data_x.to(device))
pre_lab = torch.argmax(output, 1)
acc = accuracy_score(test_data_y, pre_lab.cpu())
print(test_data_y)
print(pre_lab)
print('The prediction accuracy on the test set is', acc)
```

```
tensor([9, 2, 1,  ..., 8, 1, 5])
tensor([9, 2, 1,  ..., 8, 1, 5], device='cuda:0')
The prediction accuracy on the test set is 0.8837
```

Accelerated learning is evident in the steeper initial accuracy curve, which resulted in an improved test score of **0.8837**, though the diverging validation loss after epoch 10 clearly illustrates a classic **overfitting** scenario.



The matrix records a significant performance boost in difficult categories such as **Shirt** (691 correct) and **Pullover** (863 correct), confirming that the ReLU-based filters are more effective at distinguishing similar clothing silhouettes than the baseline Sigmoid version.

---

4. Model Optimization

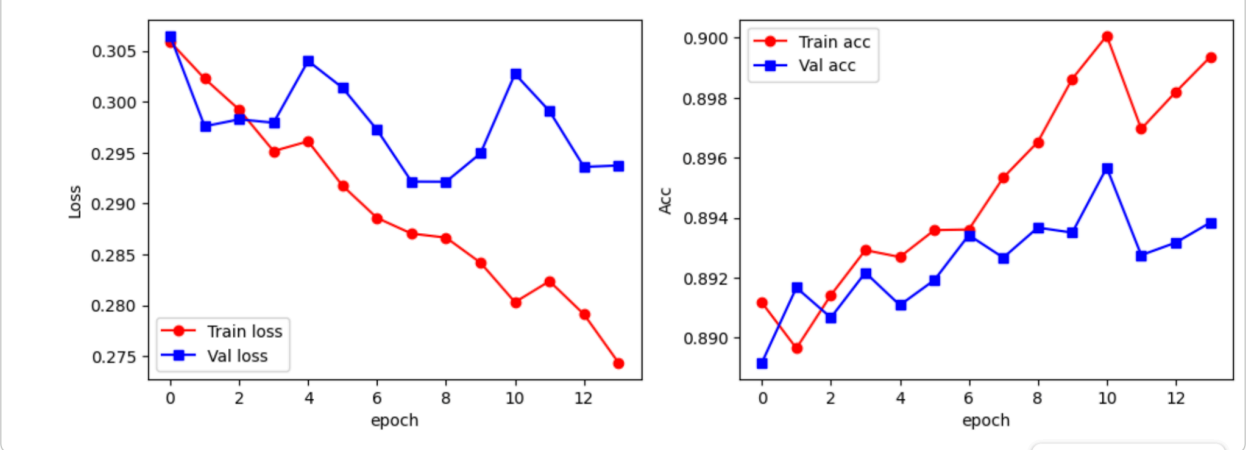
In the final iteration (Phase 3.4), three optimization techniques were applied to address the overfitting observed in Phase 3.3:

1. **Dropout (0.5 & 0.3):** Randomly deactivating neurons during training forced the model to learn more redundant and robust feature representations.
2. **Weight Decay (1e-4):** L2 regularization penalized large weights, preventing the model from over-relying on specific training samples.
3. **Early Stopping:** The training process was terminated at **Epoch 13** as validation loss began to plateau. This prevented the model from diverging and ensured the best weights were saved.

```python
# define the CNN
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 6, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.fc = nn.Sequential(
            nn.Linear(16 * 4 * 4, 120),
            nn.ReLU(),
            nn.Dropout(0.5),      # <--- Added Dropout (3.4)
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Dropout(0.3),      # <--- Added Dropout (3.4)
            nn.Linear(84, 10)
        )

    def forward(self, img):
        feature = self.conv(img)
        output = self.fc(feature.view(img.shape[0], -1))
        return output
```

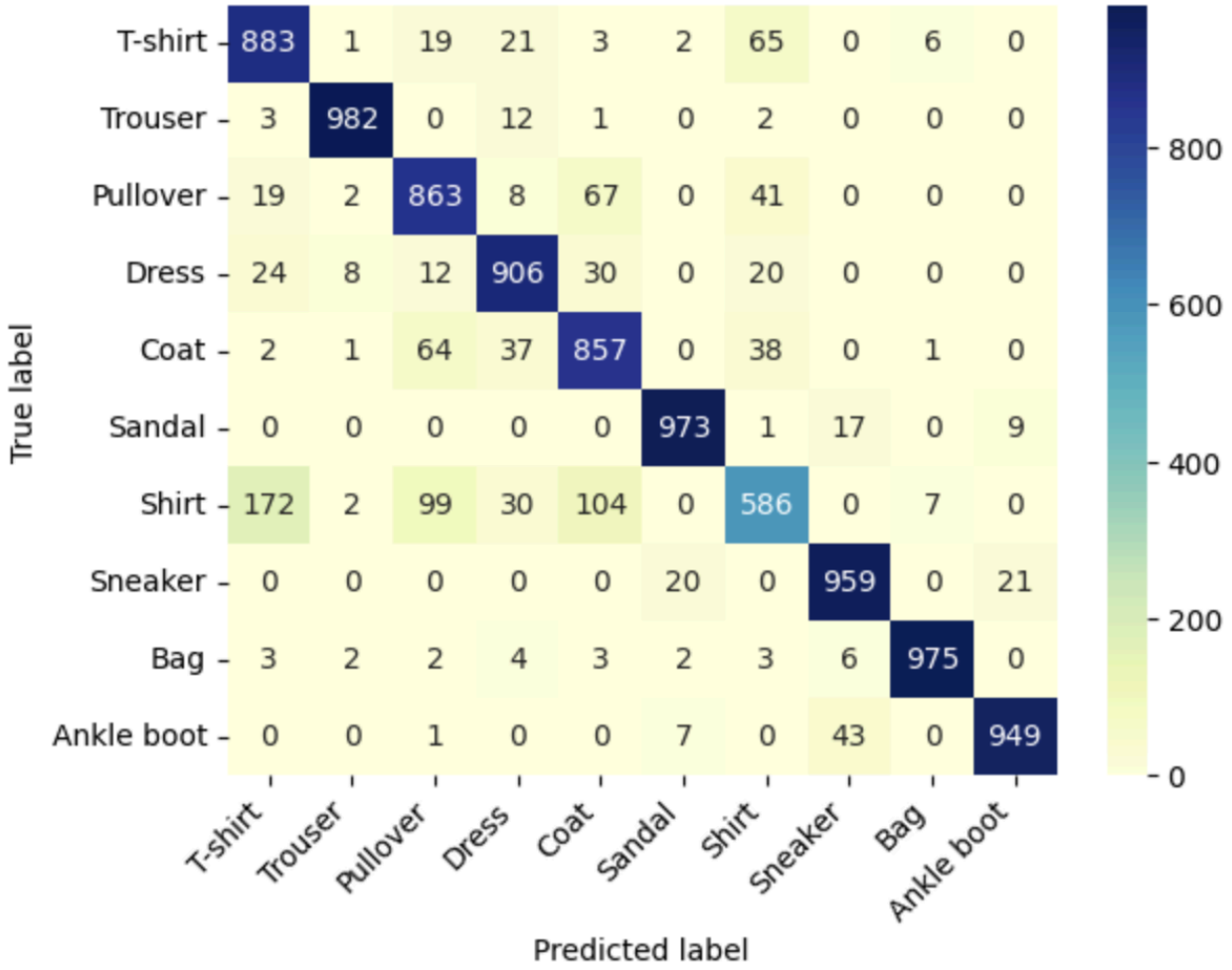
The finalized model architecture incorporates **Dropout layers** at 50% and 30% within the fully connected sequence to break neuron co-dependency and significantly enhance the network's ability to generalize to unseen data.




```python
# Test set predictions and visualize predictions
mylenet.eval()
output = mylenet(test_data_x.to(device))
pre_lab = torch.argmax(output, 1)
acc = accuracy_score(test_data_y, pre_lab.cpu())
print(test_data_y)
print(pre_lab)
print('The prediction accuracy on the test set is', acc)
```

```
tensor([9, 2, 1,  ..., 8, 1, 5])
tensor([9, 2, 1,  ..., 8, 1, 5], device='cuda:0')
The prediction accuracy on the test set is 0.8933
```

These training plots visualize the successful stabilization of the model, with the **early stopping** mechanism effectively terminating the process at epoch 13 to secure a peak test accuracy of **89.33%** and prevent the divergence observed in previous iterations.



The resulting confusion matrix demonstrates superior classification performance for distinct items like **Trousers** (982 correct) and **Sandals** (973 correct), while proving that the added regularization allowed the model to reach its most robust identification of the difficult **Shirt** category

---

5. Experience Summary

This experiment demonstrated that architectural choices (pooling and activations) are as critical as the network depth itself. Key takeaways include:

- **Generalization vs. Accuracy:** High training accuracy is irrelevant if the model overfits. Regularization (Dropout/L2) is essential for bridging the gap between training and test sets.
- **Dataset Complexity:** Fashion-MNIST presents a higher challenge than MNIST due to the similarity between classes like "Shirt," "T-shirt," and "Coat," which require precise edge detection provided by MaxPool and ReLU.
- **Computational Efficiency:** Utilizing a GPU and Early Stopping optimizes the development cycle by focusing resources only on productive training epochs.